

Concurrent Hash Table Performance Across ARM Hardware Tiers

Vaiva Staugaityte & Elias Illeris Poggi

Supervised by Peter Sestoft

IT-Universitetet i København · May 26, 2026

Why Does This Matter?

BACKGROUND

Does the performance hierarchy of concurrent hash maps transfer from high-end to resource-constrained ARM hardware?



ARM-based embedded hardware shares an ISA with our HPC node — but not cores, cache, or bandwidth. **Do the same algorithms win?**

Two ARM Machines, One Large Gap

BACKGROUND

■ both aarch64 ■

Raspberry Pi 5

Architecture	ARM Cortex-A76
Physical cores	4
Clock speed	~2.4 GHz
L3 cache	2 MB
DRAM	8 GB LPDDR4X
Threads tested	1 · 2 · 4 · 8

NVIDIA DGX Spark (HPC)

Architecture	ARMv9.2-A (aarch64)
SoC	NVIDIA GB10
Physical cores	20 (10×X925 + 10×A725)
L3 cache	24 MiB aggregate
DRAM	128 GB LPDDR5X
Mem bandwidth	~273 GB/s

8 Implementations · 3 Families

EXPERIMENTS

COARSE

*One lock
serialises all*

SynchronizedMap

Single global lock

FINE-GRAINED

*32 independent
stripe locks*

StripedMap

reads + writes lock

StripedMapPadded

+ cache-line padding

StripedWriteMap

reads lock-free

StripedWriteMapPadded

+ cache-line padding

StripedLevelWriteMap

per-stripe resize

LOCK-FREE

*No locks —
CAS only*

HashTrieMap

CAS-based Ctrie

JDK

*Fine-grained locks
+ CAS internally*

WrapConcurrentHashMap

production baseline

False Sharing & Cache-Line Padding

EXPERIMENTS

Without padding



Thread A writes lock 0 →

entire cache line invalidated on every other core
— even cores working on lock 1, 2, 3

This is **false sharing**: cores pay coherence traffic for data they don't share.

With padding (stride = 16 slots)



Thread A writes lock 0 →

only lock 0's cache line affected
lock 1, 2, 3 untouched on other cores

Effect is strongest at **high thread counts** on many-core hardware: 64 simultaneous writers → constant coherence traffic without padding.

18 Workload Configurations per Implementation

EXPERIMENTS

$$3 \times 2 \times 3 = 18$$

read ratios × key ranges × distributions

Read ratio

- 0.8 read-heavy
- 0.5 balanced
- 0.2 write-heavy

Tests whether lock-free reads pay off where it matters

Key range

- 1,000 fits in RPi L3
- 1,000,000 exceeds RPi L3

Separates contention-bound from memory-bound regimes

Distribution

- Uniform
- Zipfian-0.5 mild skew
- Zipfian-0.99 heavy skew

Real workloads are skewed — Zipf's law holds broadly

JMH: Getting Honest Numbers from the JVM

EXPERIMENTS

The JVM's JIT compiler will optimise away your benchmark unless you stop it.

Warmup (5 × 2 s)

Let the JIT compile and stabilise before recording anything.

Fork (2 fresh JVMs)

JIT profiles don't bleed between implementations.

Blackhole

Forces the compiler to treat return values as observable — prevents `get()` from being eliminated as dead code.

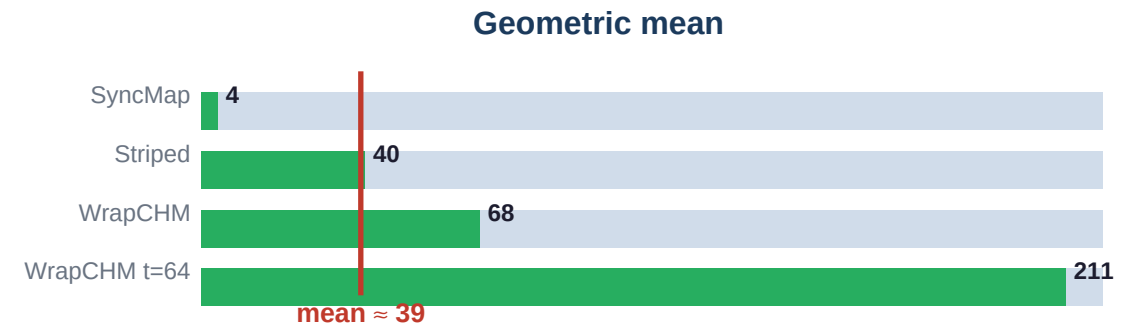
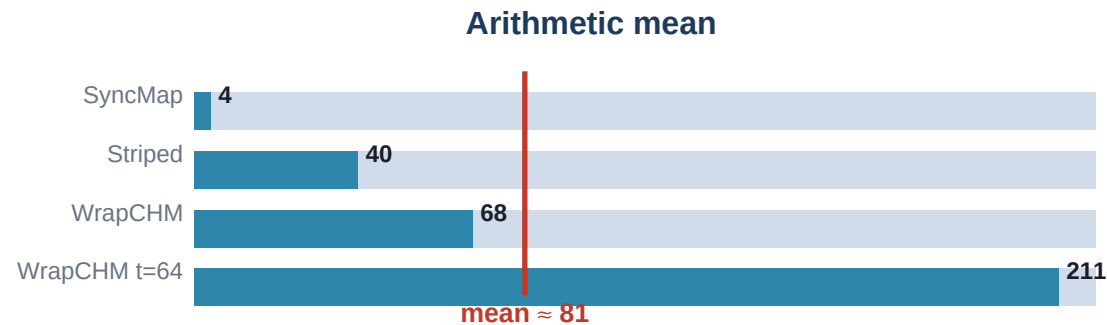
Scope.Benchmark vs Scope.Thread

One shared map across all threads (correct concurrent model). Each thread gets its own `Random`.

Summarising Across Configurations: Geometric Mean

EXPERIMENTS

Our throughput values span **two orders of magnitude**. Arithmetic mean is dominated by the fast end and masks the slow end.

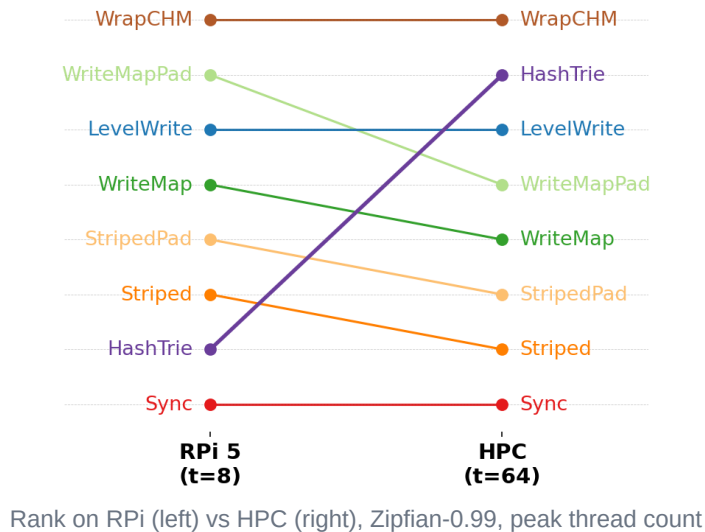


Geometric mean gives **equal weight to equal relative changes**. A 2× improvement is a 2× improvement regardless of the baseline. Formally: $\exp(\text{mean}(\log(x_1) \dots \log(x_n)))$.

Finding 1 — Rankings Transfer Across Hardware

FINDINGS

Implementation rank by platform — Zipfian-0.99, peak thread count
Rank 1 = highest throughput. A crossing line indicates rank changed across hardware.



The hierarchy holds.

WrapCHM → #1 on both

SyncMap → last on both

HashTrieMap — the exception

A crossing line in the chart means rank changed. Most lines run parallel — the ordering is stable.

HashMap: #7 on RPi · #2 on HPC

Same code. Same benchmark. Different hardware.

Why it fails on RPi 5

Pointer-chasing

INode→CNode→SNode hops frequently
miss the 2 MB L3 cache

GC pressure

Every mutation allocates new nodes.
GC competes with 4 benchmark threads.

Why it excels on HPC

24 MiB L3 cache

Trie nodes stay resident.
Pointer-chasing cost vanishes.

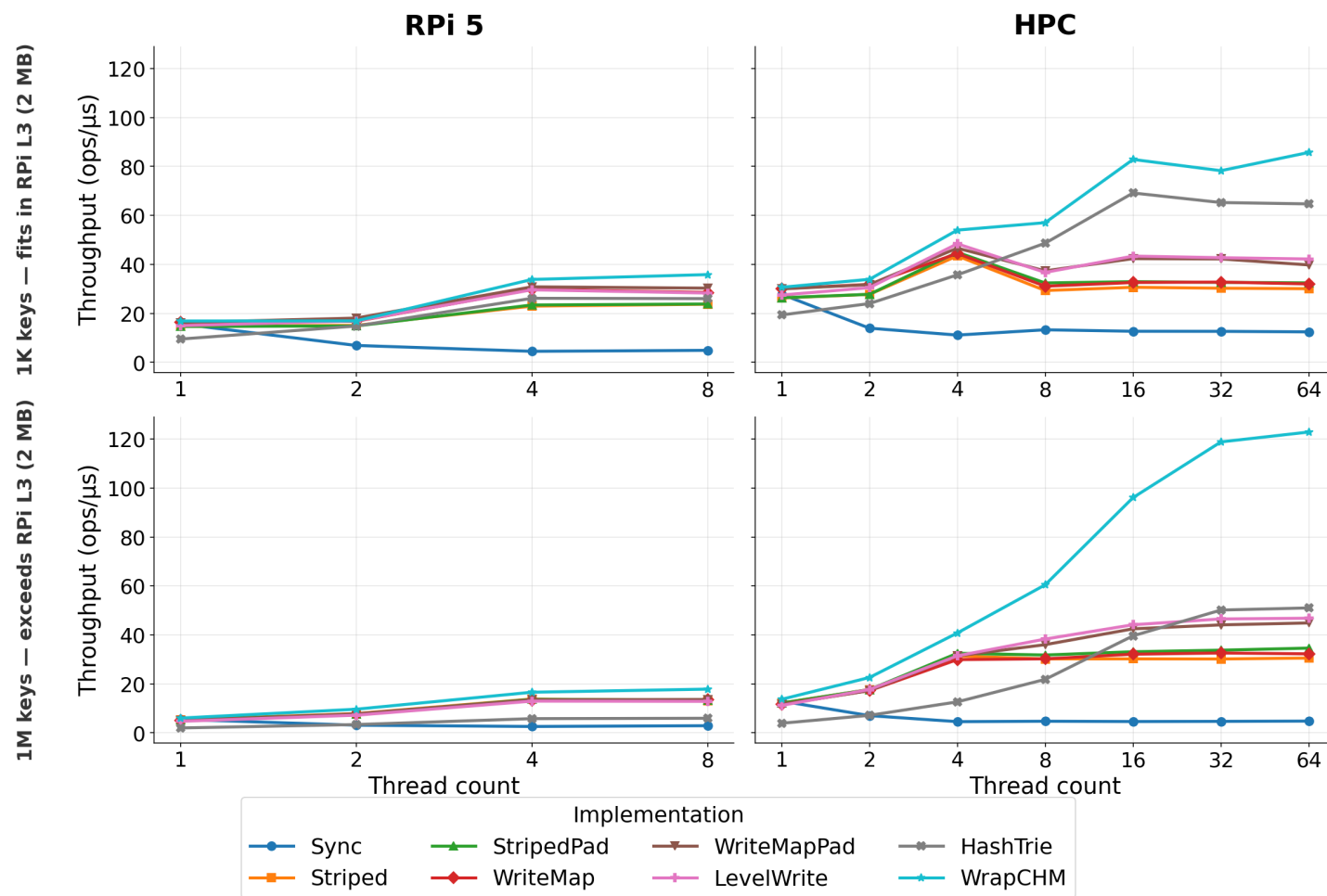
20 cores

GC competes less with benchmark threads.
Lock-free CAS scales at high thread counts.

Finding 3 — The Hardware Ceiling

FINDINGS

Thread-count scaling — geometric mean throughput (ops/μs)
Top row: 1K keys (fits in RPi L3). Bottom row: 1M keys (spills out of RPi L3).
A larger drop from top to bottom on the RPi than on the HPC indicates the cache size is the bottleneck.



Thread-count scaling (ops/μs). Top: 1K keys. Bottom: 1M keys.

RPi saturates at 4 threads.

Every implementation flatlines.

Algorithm choice barely matters.

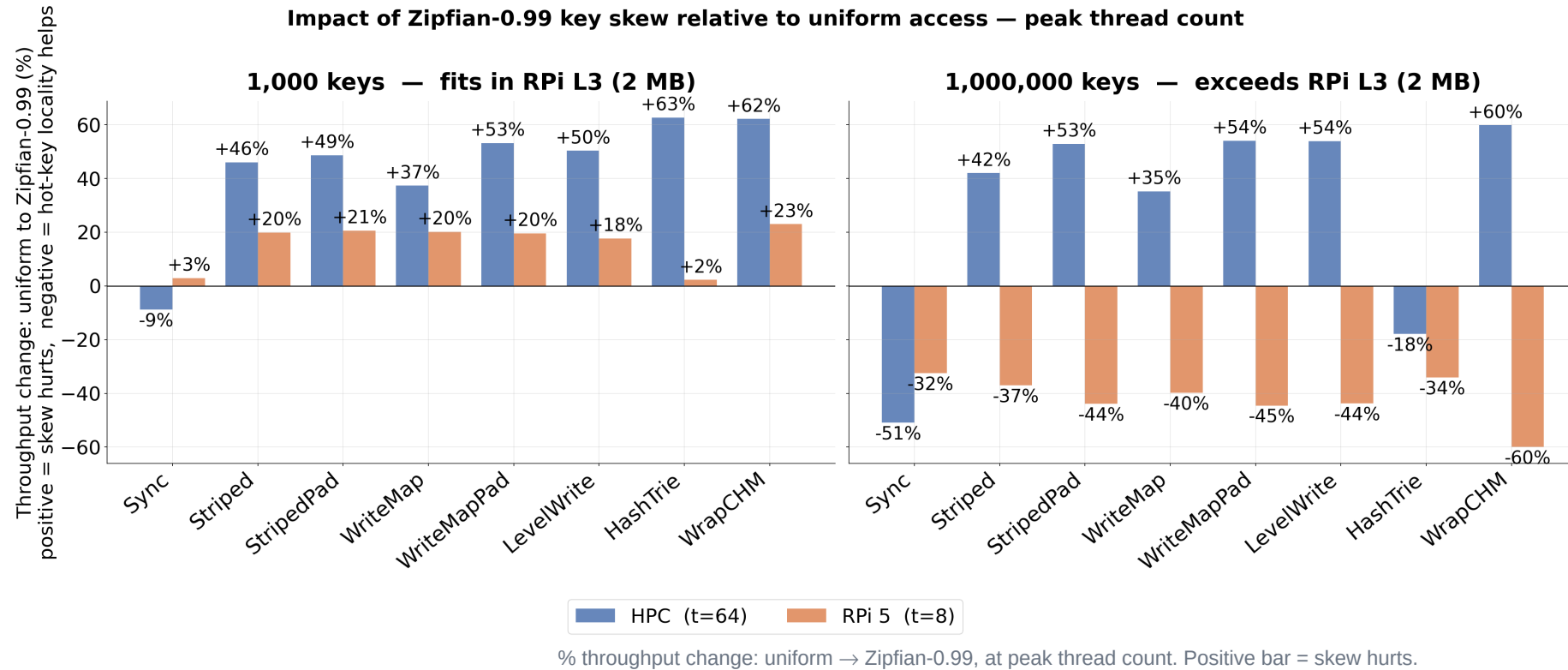
→ Use the simplest correct option.

HPC keeps scaling to t=64.

Algorithm choice matters here.

Finding 4 — Same Skew, Opposite Effects

FINDINGS



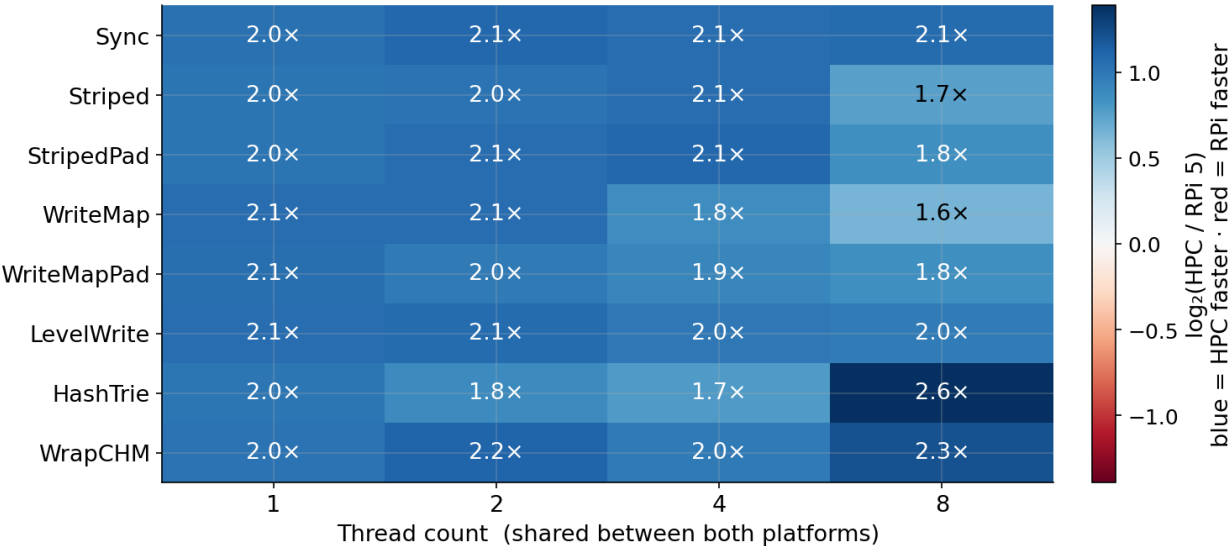
1K keys (in RPi L3): hot keys pile onto the same stripe lock. Skew hurts both platforms — HPC more, because more threads contend.

1M keys (exceeds RPi L3): RPi benefits from skew — hot keys stay in L1/L2. HPC is hurt — cache was already warm, skew only adds contention.

Finding 5 — How Much Faster Is the HPC?

FINDINGS

Hardware advantage: HPC vs RPi 5
Each cell: speedup multiplier, geometric mean across 18 workload configurations.
Colour encodes $\log_2(\text{HPC}/\text{RPi})$ — equal steps = equal relative differences.



$\log_2(\text{HPC} / \text{RPi})$ speedup. Geometric mean across 18 configurations.

~2x

faster at matched thread counts

- Clock: ~4.0 GHz vs ~2.4 GHz
- Bandwidth: LPDDR5X vs LPDDR4X
- No algorithmic outliers in heatmap

At low thread counts, the gap is a hardware property, not an algorithmic one.

Few-core · small-cache

(RPi 5 profile)

- ✓ Use StripedWriteMap
- ✗ Avoid HashTrieMap
- Saturates at #cores — don't overengineer

Many-core · large-cache

(HPC/server profile)

- ✓ WrapCHM or HashTrieMap
- ✓ Padded variants reduce false sharing
- ✗ Never SynchronizedMap in production

No universally optimal map. The right choice depends on core count and last-level cache size — both knowable at runtime.

Hardware-level profiling

Run perf stat counters — convert our inferences about cache misses and CAS retries into direct evidence.

Latency distribution

We measured throughput. Tail latency matters for real-time applications. JMH's SampleTime mode would capture it.

Hardware-adaptive implementation

Query availableProcessors() and LLC size at startup, select the backing implementation automatically.

Broader hardware coverage

BeagleBone Black (no L3 at all), single-core devices, and a systematic literature review.